Experiment No. 8

Aim
To design and implement a secure user authentication system using PHP sessions, password encryption, and database validation to manage login and logout functionality.

Software Required
• Text Editor (VS Code / Notepad++)
• Web Browser (Chrome / Firefox)
• XAMPP / WAMP Server
• PHP
• MySQL Database

Theory
User authentication is an essential feature in web applications that ensures only authorized users can access protected resources. A login system verifies user credentials such as username and password before granting access.
PHP provides session handling mechanisms that allow storing user data across multiple pages. A session is created using session_start() and remains active until it is destroyed or the browser is closed.
In modern web applications, security is enhanced using:

1. Password Hashing
Passwords are not stored as plain text. Instead, they are encrypted using:
password_hash()
This ensures that even if the database is compromised, the actual passwords remain secure.

2. Password Verification
During login, the entered password is verified using:
password_verify()
This compares the entered password with the hashed password stored in the database.

3. Prepared Statements
Prepared statements are used to prevent SQL injection attacks by separating SQL logic from user input.

4. Session Management
After successful login:
- A session is created
- User information is stored
- Access to protected pages is granted

When the user logs out:

- Session is destroyed
- User is redirected to login page

This experiment demonstrates a secure authentication system with modern UI and security practices.

Session Handling Syntax

1.Start Session

```
session_start();
```

2.Store Session Data

```
$_SESSION['user'] = $username;
```

3.Access Session Data

```
echo $_SESSION['user'];
```

4.Destroy Session

```
session_destroy();
```

Database Creation

```
CREATE DATABASE login_db;
USE login_db;
CREATE TABLE users(
id INT AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(100),
password VARCHAR(255)
);
```

Procedure

1. Install and start XAMPP/WAMP server.
2. Start Apache and MySQL services.
3. Open phpMyAdmin and create database login_db.
4. Create a users table.
5. Create database connections using PHP.
6. Design login and registration forms using Bootstrap.
7. Implement password hashing during registration.
8. Implement login validation using password verification.
9. Use sessions to manage user authentication.
10. Implement logout functionality and test the system.

Program Code

1)Database Connection:db.php

```
<?php
$conn = new mysqli("localhost","root","","login_db");
if($conn->connect_error){
die("Connection Failed");
}
```

```
?>
```

2)Login Page (UI):index.php

```
<!DOCTYPE html>
<html>
<head>
<title>Login System</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
<style>
body{
background:linear-gradient(135deg,#6366f1,#3b82f6);
height:100vh;
display:flex;
justify-content:center;
align-items:center;
}
.login-card{
width:350px;
background:white;
padding:30px;
border-radius:12px;
box-shadow:0 10px 25px rgba(0,0,0,0.2);
}
</style>
</head>
<body>
<div class="login-card">
<h3 class="text-center mb-4">Login</h3>
<form action="login.php" method="POST">
<input type="text" name="username" class="form-control mb-3" placeholder="Username"
required>
<input type="password" name="password" class="form-control mb-3" placeholder="Password"
required>
<button class="btn btn-primary w-100">Login</button>
</form>
<?php if(isset($_GET['error'])){ ?>
<div class="alert alert-danger mt-3">Invalid Credentials</div>
<?php } ?>
</div>
</body>
</html>
```

3) Registration Page:register.php

```
<?php include 'db.php'; ?>
<!DOCTYPE html>
<html>
<head>
<title>Register</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
<div class="container mt-5">
<div class="card p-4">
<h3>Register</h3>
<form method="POST">
<input type="text" name="username" class="form-control mb-3" placeholder="Username">
<input type="password" name="password" class="form-control mb-3"
placeholder="Password">
<button class="btn btn-success">Register</button>
</form>
</div>
</div>
</body>
</html>
<?php
if(isset($_POST['username'])){
$username=$_POST['username'];
$password=password_hash($_POST['password'],PASSWORD_DEFAULT);
$stmt=$conn->prepare("INSERT INTO users(username,password) VALUES(?,?)");
$stmt->bind_param("ss",$username,$password);
$stmt->execute();
echo "<script>alert('User Registered')</script>";
}
?>
```

4) Login Logic:login.php

```
<?php
session_start();
include 'db.php';
$username = trim($_POST['username']);
$password = trim($_POST['password']);
$stmt = $conn->prepare("SELECT * FROM users WHERE username=?");
$stmt->bind_param("s",$username);
$stmt->execute();
$result = $stmt->get_result();
```

```
if($result->num_rows == 1){
$user = $result->fetch_assoc();
if(password_verify($password, $user['password'])){
$_SESSION['user'] = $username;
header("Location: dashboard.php");
}else{
header("Location:index.php?error=1");
}
}else{
header("Location:index.php?error=1");
}
?>
```

5)Dashboard Page:dashboard.php

```
<?php
session_start();
if(!isset($_SESSION['user'])){
header("Location:index.php");
}
?>
<!DOCTYPE html>
<html>
<head>
<title>Dashboard</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
<div class="container mt-5 text-center">
<div class="card p-4">
<h2>Welcome <?php echo $_SESSION['user']; ?></h2>
<a href="logout.php" class="btn btn-danger mt-3">Logout</a>
</div>
</div>
</body>
</html>
```

6)Logout:logout.php

```
<?php
session_start();
session_destroy();
header("Location:index.php");
?>
```

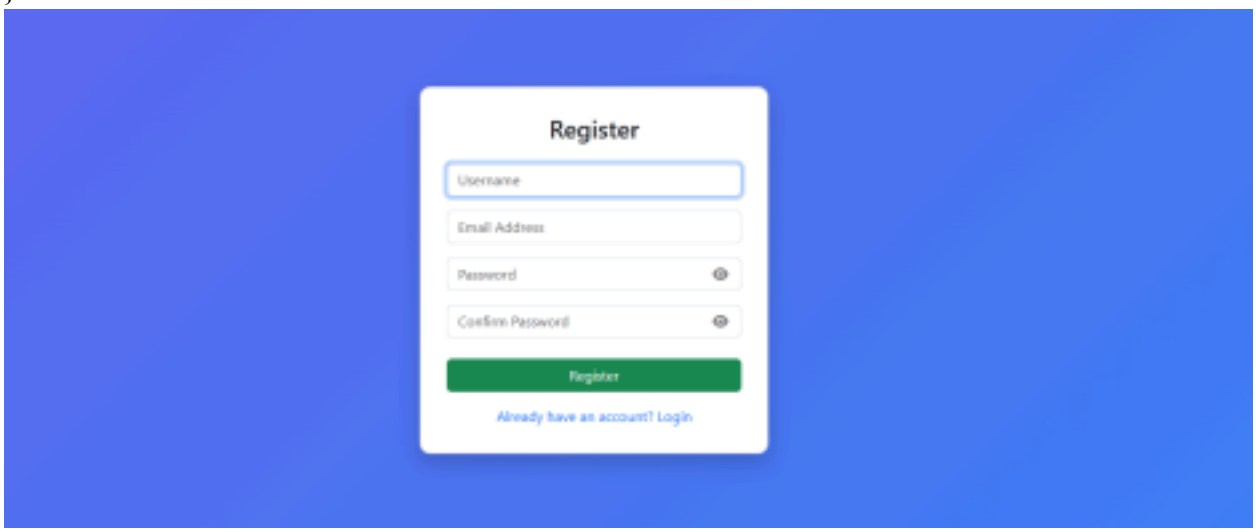
Student Tasks:

Task 1: Add Password Confirmation

Modify the registration form to include a **"Confirm Password"** field and ensure both passwords match before submission.

Code Changed in register.php: Added a "Confirm Password" input field and logic to verify both strings match before communicating with the Database.

```
php
// PHP validation logic
$password = $_POST['password'];
$confirm = $_POST['confirm_password'];
if($password !== $confirm) {
 $error = "Passwords do not match.";
} else {
 // Proceed to Database Insertion...
}
```



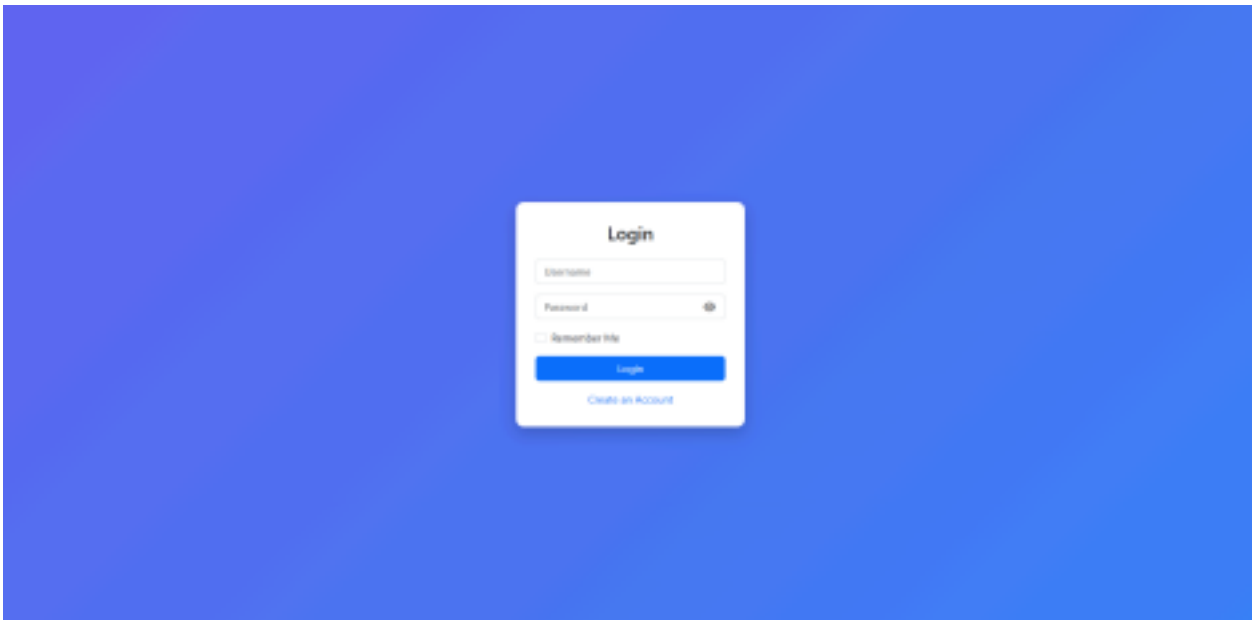
Task 2: Implement Session Timeout

Automatically log out the user after **5 minutes of inactivity**.

Code Changed in login.php & dashboard.php: Automatically tracks UNIX timestamps and signs the user out cleanly if $timeout_duration goes over 300 seconds (5 minutes).

```
php
// dashboard.php
session_start();
$timeout_duration = 300;
if(isset($_SESSION['last_activity']) && (time() -
$_SESSION['last_activity']) > $timeout_duration) {
 session_unset();
 session_destroy();
 setcookie('remember_user', '', time() - 3600, "/");
 header("Location: index.php?error=timeout");
```

```
 exit();
}
$_SESSION['last_activity'] = time(); // refreshes timer if active
```



Task 3: Add Show/Hide Password Toggle

Add a button/icon to toggle password visibility in login and registration forms. Code Changed in index.php & register.php: Implemented an interactive eye icon logic to switch type="password" to type="text".

html

```
<div class="password-container mb-3 position-relative">
 <input type="password" id="login_password" name="password"
class="form-control" placeholder="Password" required>
 <i class="fa fa-eye toggle-password"
onclick="togglePassword('login_password', this)"></i>
</div>
<script>
 function togglePassword(inputId, icon) {
 var input = document.getElementById(inputId);
 if (input.type === "password") {
 input.type = "text";
 icon.classList.remove("fa-eye");
 icon.classList.add("fa-eye-slash");
 } else {
 input.type = "password";
 icon.classList.remove("fa-eye-slash");
 icon.classList.add("fa-eye");
```

```
}
}
</script>
```

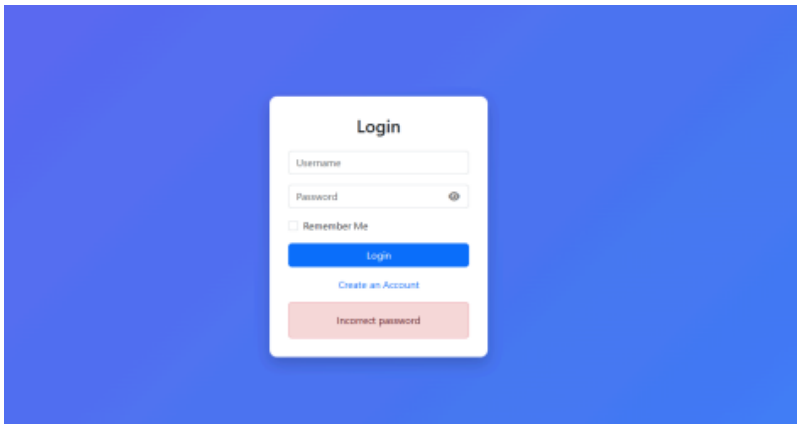
Task 4: Display Login Error Messages Properly

Instead of redirecting blindly, display specific error messages:

- “User not found”
- “Incorrect password”

Code Changed in login.php & index.php: Passed unique arguments to URLs explicitly describing the error instead of blinding redirecting.

```
php
// Login loop handling
if($result->num_rows == 1){
 if(password_verify($password, $user['password'])){
 // Valid Login
 } else {
 header("Location:index.php?error=incorrect_password");  }
 } else {
 header("Location:index.php?error=user_not_found");
 }
```



Task 5: Add Logout Confirmation

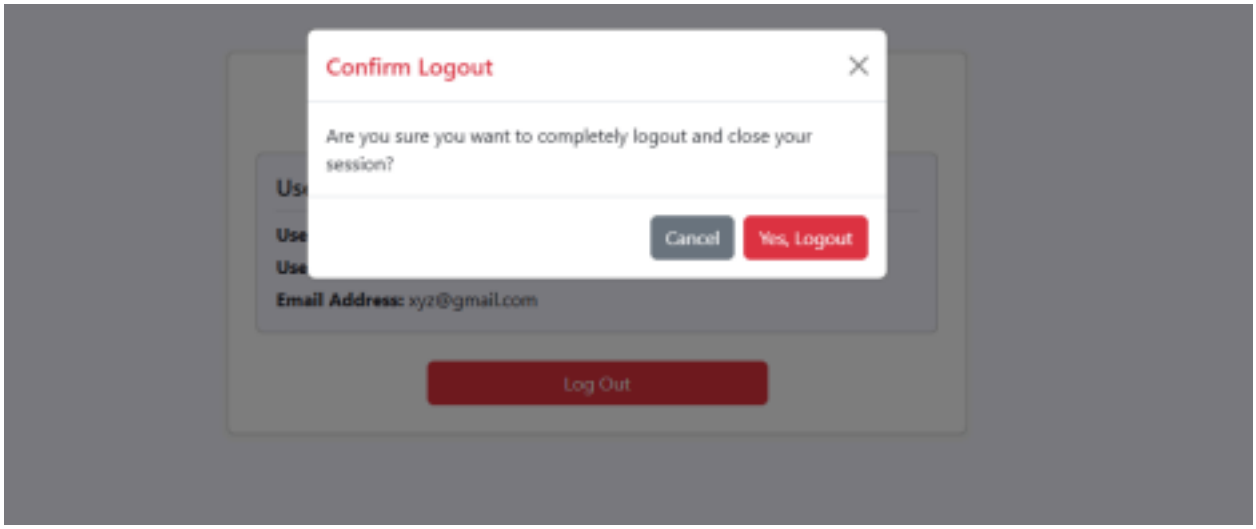
Before logging out, show a confirmation dialog:

"Are you sure you want to logout?"

Code Changed in dashboard.php: Swapped a standard href link with a Bootstrap Modal dialog that requires users to click an explicit confirmation button to logout.

html

```
<button type="button" class="btn btn-danger w-50 mx-auto"
data-bs-toggle="modal" data-bs-target="#logoutModal">
 Log Out
</button>
<!-- Triggers Modal box asking: "Are you sure you want to completely logout and close
your session?" -->
```

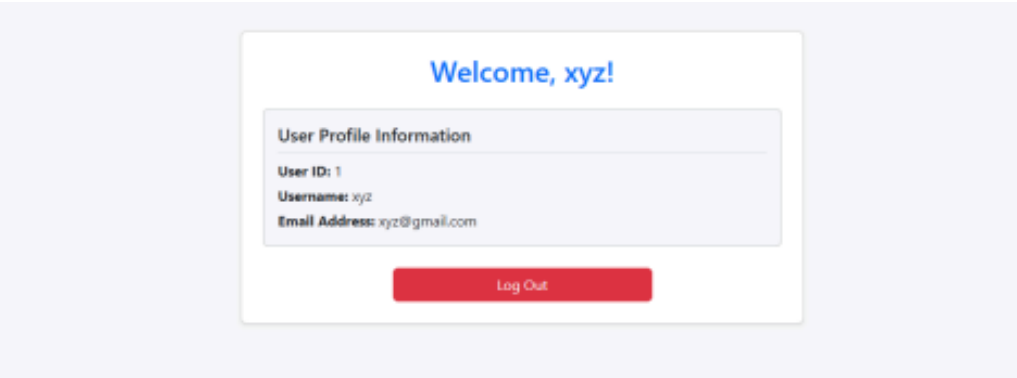


Task 6: Add User Profile Section

Get and Display additional user details (e.g., username, ID,email) on the dashboard page.

Code Changed in db.php, register.php & dashboard.php: Included an email field when structuring the table and extracted data attributes over fetch_assoc(). php

```
$username = $_SESSION['user'];
$stmt = $conn->prepare("SELECT id, username, email FROM users WHERE
username=?");
$stmt->bind_param("s", $username);
$stmt->execute();
$userDetails = $stmt->get_result()->fetch_assoc();
```



Task 7: Implement “Remember Me” Feature

Allow users to stay logged in using cookies.

Code Changed in login.php, logout.php & index.php: Implemented robust cookie hashes spanning identical strings for 30 days. Auto-redirects a visiting user if they have the token saved locally.

```
php
// login.php setup
if(isset($_POST['remember'])) {
 // Set for 30 days securely into browser
 setcookie('remember_user', $username, time() + (86400 * 30), "/"); }
// index.php auto-login logic
if(isset($_COOKIE['remember_user'])) {
 $_SESSION['user'] = $_COOKIE['remember_user'];
 $_SESSION['last_activity'] = time();
 header("Location: dashboard.php");
 exit();
}
```